PG OS

THE AI-NATIVE OPERATING SYSTEM FOR CO-LIVING

Updated Project Report, Product Specification & Technical Blueprint

A conversational, multimodal AI layer for residents, PG owners and wardens

Prepared for: Meta Global AI Developer Hackathon
Document status: Proposed concept and implementation blueprint
Revision: 2.0 | **Date:** October 30, 2026

Document Control

This report updates the original PG Booking & Management Platform proposal into PG OS. It is a product and technical proposal for a hackathon prototype; it is not evidence that the system has already been implemented or production-validated.

Field	Value
Product	PG OS — AI-Native Operating System for Co-Living
Revision	2.0
Date	October 30, 2026
Source baseline	PG Booking & Management Platform, v1.0, 19 pages
Current status	Concept / proposed build
Primary audience	Residents, PG owners and wardens
Primary differentiator	Conversational, multimodal and controlled agentic operations

Contents

1. Executive Summary
2. Product Vision and Positioning
3. Problem Statement
4. Goals, Non-Goals and Success Measures
5. Users and Core User Journeys
6. Core Product Capabilities
7. AI-Native Experience and Meta Model Integration
8. Agent Orchestration and Tool Design
9. System Architecture
10. Data Model and Database Design
11. Booking and Operational Workflows
12. WhatsApp and Voice Channel Strategy
13. Security, Privacy and Responsible AI
14. MVP Scope and Build Priorities
15. Suggested Hackathon Demonstration
16. Testing and Evaluation Plan
17. Deployment and Operations
18. Delivery Roadmap
19. Risks and Mitigations
20. Original Proposal: What Is Retained and What Changes
21. Conclusion

Appendix A. Suggested API Surface

Appendix B. Demo Dataset Suggestions

Appendix C. Sources and Reference Notes

1. Executive Summary

PG OS is an AI-native operating system for Paying Guest (PG) accommodation and co-living businesses. It connects residents, property owners and wardens through a conversational interface that can understand text, voice and images, retrieve verified operational data, and perform approved actions through secure backend tools.

The original PG Booking & Management Platform proposal established a strong operational foundation: property catalogues, room- and bed-level inventory, booking workflows, tenant records, rent tracking, WhatsApp communication and reporting. PG OS retains those core business capabilities and changes the primary interaction model. Instead of making users navigate multiple forms and dashboards for every task, it lets them ask for what they need in natural language.

For residents, PG OS can help discover suitable accommodation, check genuine availability, submit booking requests, understand stay information and report maintenance problems using a message, voice note or photograph. For owners and wardens, it can answer operational questions, summarize occupancy and outstanding rent, surface unresolved maintenance issues and coordinate follow-up work.

The AI model does not own business data and does not directly modify the database. It interprets requests and selects from a controlled set of backend tools. The backend remains the source of truth for prices, availability, bookings, rent and permissions. Sensitive actions require explicit confirmation or human approval.

The hackathon version is deliberately smaller than the original production proposal. It focuses on one responsive web application, seeded Chennai PG data, a role-aware AI orchestrator, property discovery, booking requests, owner insights and image-based maintenance ticket creation. WhatsApp and voice can be added if time and API access allow.

2. Product Vision and Positioning

Product name: PG OS — The AI-Native Operating System for Co-Living

An intelligent digital workforce that connects PG owners, wardens and residents through natural conversations.

PG OS is not simply a PG management dashboard with a chatbot attached. It is a conversational operating layer for physical accommodation businesses. AI becomes the primary interface and workflow coordinator, while reliable domain services perform the actual transactions.

The product is designed for small and mid-sized PG operators who often manage multiple properties through phone calls, WhatsApp chats, spreadsheets and paper registers. It also improves the resident experience by making discovery and ongoing communication simpler.

3. Problem Statement

PG operations are fragmented across manual registers, spreadsheets, personal messaging accounts, calls and disconnected payment records. This creates repeated work and makes it difficult to maintain a reliable view of room availability, booking status, rent dues and maintenance.

Area	Current friction	Resulting impact
Discovery and leads	Repeated calls, delayed replies, photos and prices shared manually	Prospective residents abandon enquiries or receive inconsistent information
Bed inventory	Availability maintained in registers or spreadsheets	Stale availability and risk of double booking
Booking workflow	Requests, approvals and move-in details spread across chats	No consistent audit trail or clear status
Rent and deposits	Cash, bank transfers and manual receipt records	Unclear balances, reconciliation effort and disputes
Maintenance	Residents explain issues through unstructured messages or calls	Issues are missed, misclassified or not followed up
Owner visibility	Reports prepared manually from different sources	Slow answers about occupancy, dues and operational bottlenecks

The opportunity is not only to digitize these records. It is to reduce the effort required to interact with them. A resident should be able to say, “Find a men’s PG near Sholinganallur under ■9,000 with food,” and receive results based on current property data. An owner should be able to ask, “Which property has the highest vacancy this month?” and receive an answer derived from authorized records.

4. Goals, Non-Goals and Success Measures

Product goals

- Make accommodation discovery and resident support conversational and accessible.
- Provide accurate, database-backed answers for availability, booking status, rent and occupancy.
- Allow users to report maintenance issues using text, voice or images.
- Reduce repetitive administrative work through controlled AI-assisted workflows.
- Preserve clear ownership, authorization, confirmation and auditability for every action.

Hackathon non-goals

- Building separate native iOS and Android applications.
- Launching a full payment settlement or accounting platform.
- Collecting and storing real Aadhaar or other government identity documents in the demo.
- Implementing production-grade franchise billing, IoT locks or biometric attendance.
- Claiming business impact or performance targets before measuring them.

Evaluation measures

Measure	How it will be evaluated
Property retrieval accuracy	Whether returned properties satisfy the stated location, budget, gender and amenity constraints
Availability correctness	Whether every displayed bed status matches the backend source of truth
Booking task completion	Whether a user can progress from search to a pending request with clear confirmation
Tool execution reliability	Percentage of valid requests that call the correct tool with valid arguments
Multimodal issue understanding	Whether the system extracts a useful issue category and description from a maintenance image or voice note
Authorization	Whether users are prevented from reading or changing data outside their role and property scope
Transparency	Whether the assistant clearly communicates unavailable information, uncertainty and pending approval

5. Users and Core User Journeys

Resident / prospective tenant

1. Describe accommodation needs in natural language.
2. PG OS extracts search constraints and queries the property service.
3. The system presents matching properties with real prices, amenities and current availability.
4. The resident selects a property/bed and reviews the move-in date, rent, deposit and house rules.
5. After explicit confirmation, the backend creates a pending booking request and returns a reference.
6. The resident can ask for booking status or report a maintenance issue later.

Property owner

1. The owner signs in through an authenticated account linked to permitted properties.
2. The owner asks a question such as “Show occupancy and outstanding rent for this month.”
3. The orchestrator selects the relevant read-only reporting tools.
4. The backend checks property scope and returns aggregates.
5. PG OS explains the result and can offer a structured summary or export.

Warden / property staff

1. The warden opens the operations inbox.
2. New maintenance issues and booking requests are grouped by property and urgency.

3. The warden reviews the issue, assigns a staff member and updates status.
4. The resident receives a status update through the web interface or an approved messaging channel.

The system must use authenticated identity and server-side role claims to establish whether a user is a resident, owner or warden. It must never accept a role supplied only in a chat message.

6. Core Product Capabilities

Capability	Hackathon MVP	Later production capability
Conversational discovery	Natural-language search over seeded properties	Location-aware search, ranking, saved preferences and verified listings
Inventory and booking	Read availability; create pending booking request	Temporary holds, approval queues, cancellations, check-in and checkout
Owner copilot	Occupancy and rent summaries for authorized demo data	Trend analysis, multi-property reporting, exports and alerts
Multimodal maintenance	Image upload plus text description; classify and create ticket	Voice notes, assignment, SLA tracking, photo history and vendor workflows
Resident support	Booking status and seeded house-rule answers	Full resident portal, notices, invoices and support history
Messaging	In-app chat; optional WhatsApp prototype	WhatsApp templates, interactive flows, delivery tracking and opt-in management

The demo should show one connected story rather than many disconnected features: a resident discovers a property, creates a booking request, reports a maintenance issue, and an owner reviews operational status.

7. AI-Native Experience and Meta Model Integration

Meta AI models are used where they add clear value to the workflow. The exact model identifier and API request format must be confirmed against the participant's current Meta Model API access and documentation before implementation. Model availability, rate limits and capabilities may vary by account and event access.

Model / service	Proposed responsibility	Implementation note
Muse Spark series	Primary reasoning and tool orchestration; interpret user intent and decide which approved backend tool to call	Use structured tool definitions; backend validates every argument and result
Muse Voice Transcribe	Optional transcription of resident voice notes	Treat transcription as user-provided input; confirm important dates, amounts and booking details
SAM 3.1	Optional visual segmentation for a specific maintenance-photo workflow	Not required for the MVP; it is not a substitute for a general-purpose issue classifier
Meta WhatsApp Cloud API	Messaging channel for resident and staff conversations	Transport and messaging integration, not the reasoning model
Muse Image	Optional generation or editing of non-essential visual assets	Not required for the core operational workflow
Muse Code	Development-time coding assistant	Not a runtime dependency of PG OS

The event's official page describes the hackathon as a virtual global event with an indie track for individual builders and a startup track for teams. It lists Muse Spark, Muse Image and SAM 3.1 through Meta Model API, alongside Meta open-weight models and Muse Code. The official page states project submissions are due November 4, 2026 at 9:00 PM ET; verify the event portal for any changes.

Model integration must be demonstrated rather than merely named. The project should show at least one real model call that interprets a request and selects a tool, plus one multimodal workflow if image or voice capability is included. If API access is unavailable, the report and demo must clearly disclose the fallback rather than imply that a Meta model powered the feature.

8. Agent Orchestration and Tool Design

The MVP should use one role-aware orchestrator with a small set of specialized backend tools, not multiple independent agents. This is easier to test, cheaper to operate and more realistic for a solo hackathon build. The orchestrator can operate in resident, owner or warden context, but permissions are enforced by the backend.

Tool catalogue

Tool	Purpose	Access / control
search_properties(location,budget,amenities,gender,move_in_date)	Find matching properties from the catalogue	Public or authenticated resident; validated filters
get_property_details(property_id)	Return verified property information and rules	Public/resident; published fields only
check_bed_availability(property_id,room_id,dates)	Read bed availability for requested dates	Resident; always query source of truth
create_booking_request(bed_id,move_in_date)	Create a pending booking request	Resident; explicit confirmation; transactional backend
get_booking_status(booking_ref)	Read status of the caller's booking	Owner check of user-to-booking relationship
create_maintenance_ticket(issue,description,image_ref)	Create a maintenance record	Authenticated resident; file validation and audit
get_occupancy_report(property_id,period)	Return occupancy aggregates	Owner/warden restricted to assigned property scope
get_pending_rent(property_scope,period)	Return rent due aggregates or authorized details	Owner/authorized staff only
update_maintenance_status(ticket_id,status)	Update ticket progress	Warden/authorized staff; audit trail
send_notification(recipient,message_template)	Send an approved transactional notification	Server-side template allowlist and opt-in rules

Tool calls should use typed schemas and strict validation. The model may propose a tool call, but the backend decides whether it is allowed and whether it succeeds. Tool outputs should be concise, structured and include stable identifiers and status fields. The model must not fabricate missing fields or infer booking/payment success from conversational context.

9. System Architecture

The recommended hackathon architecture is a modular monolith with clear domain boundaries. This avoids the deployment and debugging overhead of microservices while preserving a path to split services later.

Layer	Components	Responsibility
Experience	Responsive web app; chat panel; owner dashboard; resident view	Collect user input, display verified results and request confirmation
AI orchestration	Model client; orchestrator; prompt and tool registry; conversation context	Interpret intent, select tools, format responses and request clarification
Application services	Property, inventory, booking, tenant, maintenance and reporting modules	Apply domain rules and coordinate transactions
Data	PostgreSQL; object storage; optional Redis	Persist records, store media securely, manage temporary state and caching
Integration	Meta Model API; WhatsApp Cloud API; optional payment provider	External AI and messaging/payment connectivity
Governance	Authentication; RBAC; approval service; audit log; rate limits	Protect data and constrain actions

Suggested request flow: user message → authenticated session and role context → orchestrator → model reasoning/tool selection → backend authorization and validation → domain service and database transaction → structured tool result → model response → user confirmation or next action.

Recommended implementation stack

- Frontend: React + Vite + TypeScript for a single responsive web application; use a component library or Tailwind for rapid UI delivery.
- Backend: NestJS + TypeScript REST API, organized into property, booking, maintenance, tenant and reporting modules.
- Database: PostgreSQL as the authoritative store for properties, beds, bookings, rent records and maintenance tickets.
- Redis: optional for short-lived conversation/session state, rate limiting and temporary holds; do not rely on Redis alone to prevent double booking.
- Media: S3-compatible object storage or a secure development storage service for uploaded maintenance images.
- AI: Meta Model API client using the currently documented SDK or compatible API format; isolate it behind a provider interface.

- Deployment: Docker Compose for local development; one managed app deployment for the demo if time allows.

10. Data Model and Database Design

PostgreSQL remains the source of truth. The original proposal's Property → Room → Bed hierarchy is retained and extended with tenant, booking, payment-ledger and maintenance entities. The MVP can use a reduced schema while preserving the same relationships.

Entity	Important fields	Purpose
users	id, phone/email, display_name, status, created_at	Authenticated people and system users
user_roles	user_id, role, scope_type, scope_id	Role assignment and property-level access
properties	id, name, address, city, latitude, longitude, gender_type, amenities, rules, status	PG listing and ownership scope
rooms	id, property_id, room_number, floor, room_type, base_rent, deposit_amount, status	Room-level configuration
beds	id, room_id, bed_label, status	Individual inventory unit
bookings	id, booking_ref, user_id, bed_id, move_in_date, move_out_date, status, created_at	Booking request and lifecycle
tenants	id, user_id, booking_id, checkin_date, checkout_date, emergency_contact	Resident record; keep sensitive fields minimal
rent_ledger	id, tenant_id, amount, due_date, entry_type, status, reference	Rent due, payment and adjustment records
maintenance_tickets	id, property_id, room_id, bed_id, tenant_id, category, description, image_key, priority, status, assigned_to	Maintenance workflow
audit_logs	id, actor_id, action, entity_type, entity_id, metadata, created_at	Trace important actions and approvals
conversation_sessions	id, user_id, channel, state_json, updated_at	Optional short-lived conversational context

Booking consistency rules

- A booking request is not the same as a confirmed booking. The UI and AI must use precise statuses such as pending, approved, rejected, cancelled and expired.
- Use a database transaction and a database-enforced constraint or locking strategy to prevent overlapping confirmed allocations for the same bed and date range.
- If temporary holds are used, store hold expiry and recover expired holds with a background job. Redis may coordinate contention but the database must enforce final consistency.
- Use idempotency keys for booking creation so retries do not create duplicate requests.
- Re-check availability immediately before creating or approving a booking. Never trust availability cached in a prior chat turn.

The final production schema should also include indexes for property location/filter queries, foreign keys, check constraints, created/updated timestamps and soft deletion rules where legally and operationally appropriate.

11. Booking and Operational Workflows

Booking request workflow

1. Resident asks for a PG and provides preferences.
2. Search service returns matching properties and current availability.
3. Resident chooses a bed and move-in date.
4. Backend performs a fresh availability check.
5. Assistant summarizes property, rent, deposit, date and booking conditions.
6. Resident explicitly confirms submission.
7. Backend creates a pending booking request transactionally and returns a booking reference.
8. Owner or warden reviews and approves/rejects the request.

9. Resident receives status through the app and, where configured, WhatsApp.

Maintenance workflow

1. Resident uploads an image and describes the problem, for example a leaking sink.
2. AI extracts a short description and suggested category such as plumbing.
3. Resident reviews or confirms the interpretation if it is unclear.
4. Backend validates file type, size and ownership, stores the image securely and creates a ticket.
5. Warden sees the ticket in the correct property queue and assigns it.
6. Status changes are audited and communicated to the resident.

Owner insight workflow

1. Owner asks a natural-language question about occupancy or outstanding rent.
2. Backend derives permitted property scope from the authenticated account.
3. Reporting service calculates the result from database records.
4. Assistant explains the figures, period and scope; it does not invent missing data.

12. WhatsApp and Voice Channel Strategy

WhatsApp is a valuable distribution channel in India, but it should be treated as a channel adapter rather than the core application architecture. The first working version should use in-app chat. WhatsApp Cloud API integration can follow once webhook configuration, business verification, message templates and opt-in requirements are understood.

- Inbound messages arrive through a verified webhook and are mapped to a user/session using a verified phone number.
- Outbound transactional messages use approved templates where required and respect user consent and messaging policies.
- Never expose private booking, rent or KYC information to an unverified phone number.
- Keep webhook signature verification, replay protection, idempotency and delivery-status handling on the backend.
- Voice notes can be transcribed where the selected Meta transcription capability is available; important dates, amounts and commitments should be repeated back for confirmation.

13. Security, Privacy and Responsible AI

PG OS handles personal information and may eventually handle identity documents, residential details and financial records. The hackathon should use synthetic data and avoid collecting real government identity documents. A production launch requires a separate privacy and security review.

Security controls

- Authentication: use a trusted identity provider or secure OTP flow; store passwords only with a modern password hash if passwords are used.
- Authorization: enforce role and property scope in backend services for every request; do not rely on prompt instructions or frontend hiding.
- Secrets: keep API keys and webhook secrets in environment-based secret storage; never expose them to the browser or model context.
- Transport and storage: use HTTPS in deployment; encrypt sensitive data and object storage; use short-lived signed media URLs.
- Input handling: validate schemas, file types and sizes; rate-limit login, uploads and AI tool calls; sanitize output where rendered.
- Audit: record actor, action, target, time, approval and outcome for bookings, rent adjustments and maintenance status changes.
- Retention: define data minimization, retention, deletion and access processes before collecting real tenant information.

AI safety and transaction controls

- The model must not directly query the database or execute arbitrary SQL.
- Never let the model mark rent as paid, issue a refund, cancel a booking or approve a tenant without the appropriate workflow and authority.
- Require explicit resident confirmation before creating a booking request.
- Require owner/warden authorization and, where appropriate, a second human approval for sensitive operational changes.

- Treat uploaded images, voice transcripts and user messages as untrusted input; do not follow instructions embedded in files or images.
- If the model is uncertain, ask a focused question or route the issue to a human instead of guessing.
- Return only the minimum personal data needed for the current task.

14. MVP Scope and Build Priorities

The original commercial proposal estimated approximately 1,000 engineering hours over 12–14 weeks for a multi-client production platform. That estimate does not describe the hackathon build. The hackathon MVP is a proof of concept with a much narrower scope and should not be presented as production-ready.

Priority	Feature	Acceptance condition
P0	Seeded PG/property/room/bed data	At least several realistic demo properties with consistent room and bed inventory
P0	Authenticated role-aware web experience	Resident and owner views are separated by server-enforced permissions
P0	Natural-language property discovery	Model extracts preferences and uses the property search tool
P0	Availability and booking request	Current availability is checked; pending request is created only after confirmation
P0	Owner operational query	Owner receives database-backed occupancy and rent summary for permitted properties
P0	Image-based maintenance ticket	Image and text produce a ticket with a useful category and traceable status
P1	WhatsApp Cloud API	Inbound/outbound message prototype with verified webhook and consent
P1	Voice input	Transcription of a short voice note with confirmation of critical details
P2	Native mobile apps, UPI payments, full KYC vault, franchise management	Explicitly deferred beyond the hackathon

A polished end-to-end experience with reliable data and permission checks is more valuable than a large feature list with incomplete workflows.

15. Suggested Hackathon Demonstration

Recommended demo duration: approximately five minutes. Use seeded records and a clean reset path so the demo does not depend on real PG owners, live inventory or payment gateways.

1. Resident discovery: “Find me a men’s PG near Sholinganallur under ■9,000 with food.” Show the model extracting constraints, calling the search tool and presenting matching properties from the database.

2. Booking request: resident selects a bed and date. The system checks live seeded inventory, summarizes the terms and asks for confirmation. After confirmation, it creates a pending booking with a reference.

3. Multimodal maintenance: resident uploads a sample photo and says “There is water leaking under my sink.” Show the model’s interpretation, the resident confirmation if needed, and a newly created maintenance ticket.

4. Owner copilot: switch to an owner account and ask “Which of my PGs has the highest vacancy, and what rent is outstanding this month?” Show an authorized database-backed summary.

5. Safety moment: demonstrate that the resident cannot access another tenant’s booking or an owner-only rent report.

The demo should visibly show the model call and tool execution, while making clear that availability, booking state and financial values come from backend data.

16. Testing and Evaluation Plan

Functional and AI test cases

- Budget and location are missing: ask a clarification rather than inventing preferences.
- No beds are available: explain that no matching bed is available and offer alternatives.
- Two residents request the same bed: ensure only one request can progress under the configured allocation rules.

- Resident asks for another tenant's booking: deny access without revealing whether the record exists.
- Owner asks about a property outside their scope: deny the request.
- User changes move-in date after availability was checked: run a fresh availability check.
- Image is blurry or unrelated: ask for a clearer image or manual description.
- Voice transcript contains an ambiguous amount/date: repeat it back for confirmation.
- Booking tool times out: do not claim success; allow safe retry using idempotency.
- User asks the AI to mark rent as paid without a verified payment record: refuse and direct to the payment/ledger workflow.

Quality metrics

- Tool selection accuracy and schema-valid tool-call rate.
- Property filter precision and availability correctness.
- Booking completion rate and duplicate-request rate.
- Authorization pass/fail results across resident, owner and warden test accounts.
- Maintenance category usefulness and ticket creation success.
- Median and p95 response latency measured on the actual demo environment.
- Clear handling of uncertainty, failures and unavailable data.

Create a repeatable test set of at least 20–30 scenarios. Record expected outcomes and actual outcomes; do not report a success percentage until the tests have been run.

17. Deployment and Operations

- Local development: Docker Compose for backend, PostgreSQL and optional Redis.
- Environment separation: local, demo/staging and production configurations with separate secrets and data.
- Database migrations: versioned migrations checked into source control; seed scripts for demo data.
- Observability: structured logs with request IDs, tool names, latency, failures and actor IDs; avoid logging sensitive message contents unnecessarily.
- Backups: automated database backups and a tested restore procedure for production.
- Reliability: graceful handling of model/API timeouts, retry limits, webhook retries and idempotent operations.
- Release: deploy a single web application and API for the hackathon; split services only when scaling or team ownership requires it.

18. Delivery Roadmap

Stage	Scope	Exit criteria
Hackathon prototype	Core discovery, booking request, owner query, image maintenance and role enforcement	A stable end-to-end demo with seeded data and repeatable test results
Pilot MVP	Real PG onboarding, verified property data, resident/owner accounts, booking approvals and maintenance inbox	Pilot properties can complete day-to-day workflows with support and monitoring
Production foundation	Privacy review, security hardening, backups, audit, operational support, WhatsApp integration	Production readiness review and signed operational policies
Expansion	Payments, invoices, multi-branch management, voice, advanced analytics and integrations	Features prioritized using pilot feedback and measurable adoption

The original proposal's broader capabilities—Flutter mobile applications, Next.js SEO portal, full admin dashboard, optional UPI gateway, PDF receipts, expanded KYC vault and franchise features—remain possible later phases. They are not required to prove the AI-native concept.

19. Risks and Mitigations

Risk	Mitigation
Model invents availability or rent	All operational facts come from backend tools; responses cite current tool results in the UI where practical
Unauthorized data access	Server-side RBAC and property scoping; negative authorization tests
Double booking	Database transaction and constraints; idempotency; fresh availability checks
Meta API access or model limits	Verify access early; isolate model adapter; disclose any fallback clearly
WhatsApp onboarding delay	Keep in-app chat as the primary demo channel
Scope too large for solo build	Prioritize P0 flows; defer native apps, payments and full KYC
Sensitive tenant data exposure	Use synthetic demo data; minimize collection; signed URLs and retention controls
Unreliable AI classification	Allow resident confirmation and manual category correction

20. Original Proposal: What Is Retained and What Changes

Retained from the original report: the Property → Room → Bed inventory model; property catalogue and search; booking request and approval lifecycle; tenant and stay records; rent ledger concepts; WhatsApp as an important communication channel; reporting; role-based access control; PostgreSQL; Redis as an optional supporting component; and secure media storage.

Changed for PG OS: the primary experience becomes conversational; AI can process text, voice and images; a role-aware orchestrator coordinates workflows; backend tools enforce domain rules; maintenance becomes a core MVP scenario rather than only a future roadmap item; owner analytics becomes question-driven; and human approval is explicit for sensitive operations.

Deferred from the original commercial scope: three separate client applications, native Flutter apps, full UPI settlement, complete KYC document vault, production AWS architecture, app-store launch, and the original 1,000-hour/■7.5 lakh commercial estimate. Those figures belong to the original full platform proposal and should not be reused as the hackathon MVP estimate.

21. Conclusion

PG OS turns a conventional PG management platform into an AI-native operating experience. It combines a practical business domain with multimodal interaction and controlled agentic workflows. Residents can discover and manage their stay through natural conversation, while owners and wardens can operate their properties with less manual coordination.

The central engineering principle is simple: AI understands and coordinates; trusted backend services decide and execute. PostgreSQL remains the source of truth, authorization is enforced outside the model, and sensitive actions require confirmation or human approval.

The recommended hackathon strategy is to build a small, working, measurable prototype that demonstrates the complete value proposition. A reliable resident-to-owner workflow will make a stronger case than a broad platform whose core operations are not yet functional.

Appendix A. Suggested API Surface

Method and route	Purpose
POST /api/v1/ai/chat	Accept authenticated user message and return assistant response/tool result
GET /api/v1/properties	Search published properties using filters
GET /api/v1/properties/:id	Fetch property details
GET /api/v1/properties/:id/availability	Check bed availability for dates
POST /api/v1/bookings	Create a pending booking request after confirmation
GET /api/v1/bookings/:ref	Read caller-authorized booking status
POST /api/v1/maintenance-tickets	Create a maintenance ticket with optional image reference
GET /api/v1/owner/reports/occupancy	Get authorized occupancy summary
GET /api/v1/owner/reports/rent-due	Get authorized rent due summary
PATCH /api/v1/maintenance-tickets/:id/status	Update ticket status for authorized staff
POST /api/v1/webhooks/whatsapp	Receive verified WhatsApp webhook events (optional phase)

Every route requires authentication where appropriate, request validation, authorization checks, structured errors and audit logging. The AI chat endpoint must not become a bypass around domain APIs.

Appendix B. Demo Dataset Suggestions

- Create 4–6 fictional PG properties across Sholinganallur, Perungudi, Thoraipakkam and Velachery, Chennai.
- Include a mix of men's, women's and co-living properties, different rent bands, food availability and amenities.
- Create multiple rooms and individual beds with occupied, available and maintenance states.
- Seed resident accounts, owner-to-property assignments, booking requests, rent ledger entries and maintenance tickets.
- Use fictional names, phone numbers and images licensed for demo use. Never use real tenant identity documents.

Appendix C. Sources and Reference Notes

Original source document: "PG Booking & Management Platform — Comprehensive Project Report & Blueprint", version 1.0, dated October 30, 2026, 19 pages. The updated report retains its core domain model and business workflows while changing the product positioning and hackathon scope.

Hackathon source: Meta Global AI Developer Hackathon official event page. The event page describes the event tracks, themes, model access and published dates. Check the page again before submission because event dates and access details may change.

Official event page: https://dev.meta.ai/events/global-hackathon

Cerebral Valley event page: https://events.cerebralvalley.ai/e/meta-ai-developer-hackathon

Meta AI Research: https://research.meta.ai/

Model capability mapping in this report is a proposed implementation plan, not a claim that every model endpoint has been tested in this project. Confirm exact model names, endpoint availability, SDK usage, pricing, rate limits and event credits in the current participant documentation.